SwiftLog Technical Report

Machine Learning Track | Mission ML05: Intelligent Logistics Prediction & Supply Chain Optimization

1. Purpose

This report documents the machine-learning workflow for four logistics tasks using the supplied SwiftLog educational datasets. The workflow includes data quality review, feature engineering, candidate algorithm comparison, held-out evaluation, limited hyperparameter tuning, permutation importance, saved model bundles, and a Streamlit prototype.

2. Data preparation

Source tables were loaded from Data/Original_Datasets. Processed copies were reviewed for row counts, duplicates, and missing cells. Related order, item, product, customer, shipment, warehouse, and return tables were joined where required.

3. Targets and feature design

Delivery delay: positive when DelayDays > 0; DelayDays and post-event delivery status are excluded from predictors. Product return: positive when an OrderID appears in Returns. Demand forecasting: weekly product quantity is the target, with lagged prior-week demand and rolling history as predictors. Customer value: next-month revenue is the target, with features summarizing the preceding three months of activity.

4. Evaluation design

Classification tasks report accuracy, precision, recall, F1, and ROC-AUC where available. Demand and customer-value tasks report MAE, RMSE, and R-squared where available. The demand and customer-value tasks use chronological train/test splits. Results should be treated as experimental.

5. Model comparison

The following table reports the available candidate algorithm metrics. Selected marks the model stored in the model bundle.

Problem	Algorithm	Selected	Tuned
Delivery Delay Classification	Random Forest (tuned)	True	True
Delivery Delay Classification	Decision Tree	False	False
Delivery Delay Classification	Logistic Regression	False	False
Product Return Classification	Decision Tree	True	False
Product Return Classification	Logistic Regression	False	False
Product Return Classification	Random Forest	False	False
Product Demand Forecasting	Random Forest Regressor (tuned)	True	True
Product Demand Forecasting	Decision Tree Regressor	False	False
Product Demand Forecasting	Linear Regression	False	False
Customer Value Prediction (Next Month)	Linear Regression	True	False
Customer Value Prediction (Next Month)	Decision Tree Regressor	False	False
Customer Value Prediction (Next Month)	Random Forest Regressor	False	False

6. Interpretation

The delivery-delay target is imbalanced, and its ROC-AUC is close to 0.5. Its F1 score should therefore not be interpreted as strong ability to distinguish delayed from on-time shipments. The return classifier also shows weak discrimination. Negative held-out R-squared for demand and customer-value models indicates worse performance than a simple mean baseline on the test periods. These results are reported transparently rather than presented as successful forecasts.

7. Feature importance and artefacts

Permutation importance is saved in Models/Permutation_Importance.csv and visualized in Models/Feature_Importance.png when available. Classification confusion matrices are saved under Models/. The selected pipelines are stored in Models/Best_Model.pkl. The Streamlit app loads this bundle for demonstrations.

8. Assumptions and limitations

The delay target is defined from DelayDays because the supplied status field may disagree with the numeric delay field. A recorded return is treated as the return label. Supplier reliability is not modelled because the available tables do not directly link supplier IDs to shipments. Customer value is defined as next-month revenue rather than lifetime value. The data covers a limited historical period and does not establish production-grade predictive performance.

9. Recommendations

Improve data capture for planned and actual delivery times; reconcile delivery status with delay days; capture supplier IDs on relevant records; maintain time-stamped inventory snapshots; define return labels consistently; collect more historical periods; evaluate models against simple baselines; and pilot predictions with human review before operational use.

10. Reproduction

Install dependencies from App/requirements.txt, then run python run_swiftlog_project.py from the project root. The script regenerates processed datasets, model comparisons, saved model bundles, evaluation figures, reports, and the presentation. The notebook reviews the outputs.